

PHOT 110: Introduction to programming

Lecture notes

Michaël Barbier

2026-04-14

Table of contents

Preface	3
1 Introduction	4
1.1 Python basics	4
1.1.1 Python program structure	4
1.1.2 The Python console	4
1.1.3 Python scripts	5
1.2 Data objects	5
1.2.1 Object type	5
1.2.2 Variables	6
1.3 Expressions & Operators	6
1.3.1 Structure of an expression	7
1.3.2 Operator precedence and brackets	7
1.4 Text objects	8
1.4.1 Operators working on text objects	8
1.4.2 Formatting strings and numbers	9
1.5 Python reserved words or keywords	9
1.6 input & output	9
1.7 Indentation	10
1.8 Import statements	10
2 Tidy and Documented code	11
2.1 Tidy Writing style	11
2.2 Comment lines	11
2.3 Multi-line comments]	11
2.4 Docstrings	12
3 Program Errors	13
3.1 Different error-types	13
3.1.1 Syntax error examples	13
3.1.2 Runtime error examples	14

Preface

This is a Quarto book.

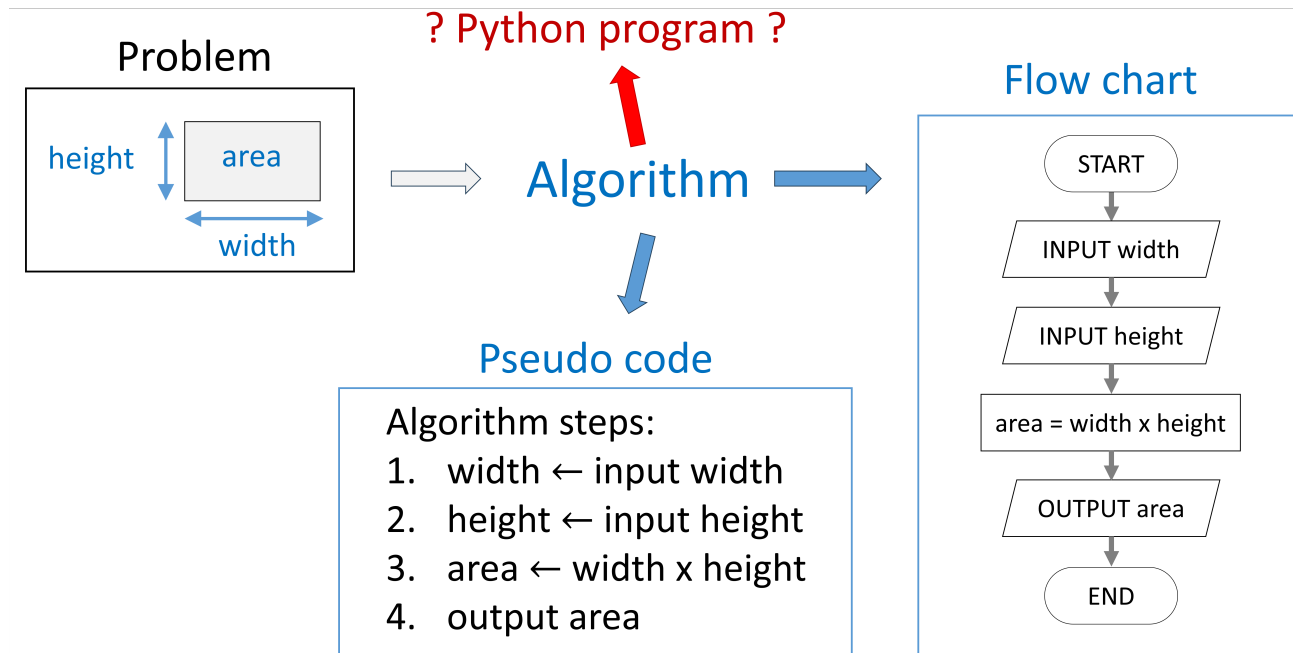
To learn more about Quarto books visit <https://quarto.org/docs/books>.

1 Introduction

This is a book created from markdown and executable code.

See Knuth (1984) for additional discussion of literate programming.

Towards Python implementations



1.1 Python basics

1.1.1 Python program structure

A Python script is a sequence of **statements** (algorithm steps), and **definitions** (functions, classes, ...)

- Definitions are evaluated
- Statements are executed

Executing a program:

- Execute statements **one by one** in the **Python Console**
- Statements **stored** as a sequence in a **script file**

1.1.2 The Python console

Execute statements one by one in the Python Console

```

1 >>> print(6)
2 6
3 >>> a = 10
4 >>>

```

Useful for:

- quick calculations
- testing the working of commands

1.1.3 Python scripts

Statements stored in a script file

```

a = 10
b = 25
prod = a * b
print(f"{a} times {b} = {prod}")

```

10 times 25 = 250

Useful for:

- running a program multiple times
- incorrect lines can easily be found and corrected
- larger programs

1.2 Data objects

- In Python everything is a data object: numbers, text, functions, class instances, etc.
- Statements perform actions on data objects

Data objects can be scalar:

Type	Description	(Example) values
bool	Boolean value	True, False
int	Integer numbers	.. , -2, -1, 0, -1, -2, ..
float	Floating point	3.56, 23e-3, 0.0079
NoneType	Indicates no object	None

1.2.1 Object type

- Every object has a type

```
print(type(6.22e23))
```

```
<class 'float'>
```

- Possible to convert between certain types: type casting

```
it_is_raining = True
print(type(it_is_raining))
an_integer = int(it_is_raining)
print(type(an_integer))
```

```
<class 'bool'>
<class 'int'>
```

- automatic type casting in some cases
 - float <= int * float
 - float <= int + float
 - float <= int / int

```
a = 4
b = 0.357
print( f"Type of (a * b) = {type(a * b)}" )
print( f"Type of (a / b) = {type(a / b)}" )
print( f"Type of (a + b) = {type(a + b)}" )
print( f"Type of (a / 25) = {type(a / 25)}" )
```

```
Type of (a * b) = <class 'float'>
Type of (a / b) = <class 'float'>
Type of (a + b) = <class 'float'>
Type of (a / 25) = <class 'float'>
```

1.2.2 Variables

A **variable** is a name bound to an **object**:

- the object is **assigned** to the variable
- In pseudo code indicated by ←

In python assignment operator is the “=” sign:

```
a_variable_name = 4.5
```

The variable can be re-assigned another value or object:

```
v = 4
v = v + 2                # v becomes 6
v = "Now v is assigned text"  # v has type str
```

1.3 Expressions & Operators

Objects can be combined by **operators** in **expressions**

Most used arithmetic operators:

symbol	description	example
**	Power	$3^{**}2 = 9$
/	Division	$5 / 4 = 1.25$
*	Multiplication	$3 * 4 = 12$
+	Addition	$5 + 7 = 12$
-	Subtraction	$6 - 9 = -3$
%	modulo	$34 \% 6 = 4$

Most used logic operators:

symbol	description	example
<, >	smaller/larger than	$5 > 4 \rightarrow \text{True}$
==	is equal to	$3 == 6 \rightarrow \text{False}$
<=, >=	smaller/larger or equal	$5 <= 5 \rightarrow \text{True}$
and	boolean AND	$\text{True and False} \rightarrow \text{False}$
or	boolean OR	$\text{True or False} \rightarrow \text{True}$
not	boolean NOT	$\text{not True} \rightarrow \text{False}$

1.3.1 Structure of an expression

An expression can be built with following rules:

- $\langle \text{expression} \rangle \stackrel{\text{def}}{=} \langle \text{object} \rangle$ (an object is an expression)
- $\langle \text{expression} \rangle \stackrel{\text{def}}{=} \langle \text{operator} \rangle \langle \text{expression} \rangle$
- $\langle \text{expression} \rangle \stackrel{\text{def}}{=} \langle \text{expression} \rangle \langle \text{operator} \rangle \langle \text{expression} \rangle$

Expressions result into values and can be assigned to variables:

```
x = 4
y = 2*x**2 + 3*x - 5
print(f"The value of y = {y}")
```

The value of y = 39

1.3.2 Operator precedence and brackets

Precedence of operators is similar to mathematics.

List of precedence of operators can be found on the python.org website: <https://docs.python.org/3/reference/expressions.html#operator-precedence>

Round brackets can be used to give priority

```
x = 2
y = (12 - x) / 10
print(f"The value of y = {y}")
```

The value of y = 1.0

1.4 Text objects

Text objects are called strings: the type is `str`. A string is defined by using single or double quotes:

```
"This is a string, single 'quotes' can be used here"  
'Here we can use double "quotes" inside it'
```

Strings can also cover multiple lines, for that we use triple quotes:

```
""" This string runs over multiple lines.  
Another line starts here.  
And another one.  
"""
```

Also triple single quotes work as well.

1.4.1 Operators working on text objects

Appending one string to another with the “+” operator:

```
a_verb = "flies"  
print("The balloon" + a_verb)  
print("The balloon" + " " + a_verb)
```

```
The balloonflies  
The balloon flies
```

Multiplying strings:

```
print(5 * "balloon ")
```

```
balloon balloon balloon balloon balloon
```

Casting a string to a number and vice versa:

```
print(5 * int("100"))  
print(10 * str(50))
```

```
500  
50505050505050505050
```


1.4.2 Formatting strings and numbers

A formatted string is a string with prefix `f` or `F`:

```
the_radius = 25
formatted_str = f"A circle with radius {the_radius} m"
print(formatted_str)
```

A circle with radius 25 m

More complex formatting:

```
a = 1/6; b = 0.0145; c = 23e-6
print(f"The product {a} x {b} = {a * b}")
# Use format {variable:No_space.No_sign_digits}
print(f"{a:8.2} x {b:8.2} = {a * b:8.2}")
print(f"{a:8.2} x {c:8.2} = {a * c:8.2}")
```

```
The product 0.16666666666666666 x 0.0145 = 0.0024166666666666667
    0.17 x    0.015 =    0.0024
    0.17 x  2.3e-05 =   3.8e-06
```

1.5 Python reserved words or keywords

- A list of words **reserved** for Python
- You cannot name variables (or functions/classes) similar

```
# List of keywords can be found in the keyword package
import keyword

print(keyword.kw_list)
```

```
['False', 'None', 'True', 'and', 'as', 'assert', 'async', 'await', 'break', 'class', 'continue', 'def', 'del', 'elif', 'except', 'finally', 'for', 'from', 'global', 'if', 'import', 'in', 'is', 'lambda', 'nonlocal', 'not', 'or', 'pass', 'raise', 'return', 'try', 'while', 'with', 'yield']
```

1.6 input & output

When not in interactive mode:

- `input()` can be used to ask user text input
- `print()` can be used to output text

```
1 # parameters
2 pizza_price = 200
3 extra_cheese_price = 20
4
5 # input accepts a string parameter
6 n_pizza = input("How many pizza's do you want: ")
7 extra_cheese = input("""Do you want extra cheese?\n
8     For yes press [1]\n
```

```

9     For no press [0]\n
10 Enter your choice"")
11
12 # Output to the user
13 print("The total cost is {int(n_pizza * (pizza_price + extra_cheese))}")

```

1.7 Indentation

- Is the amount of space preceding statements
- Indentation should be the same within a code block
- **Spaces** and **Tabs** are different
- Following code will give an `IndentationError: unexpected indent`

```

a = 4
b = 2
  c = 3
y = a * (b + c)

```

```

IndentationError: unexpected indent (3759259151.py, line 3)
Cell In[33], line 3
    c = 3
    ^
IndentationError: unexpected indent

```

1.8 Import statements

Python packages can be imported in multiple ways

```

1 # import the package with its name
2 import math
3
4 area = 3**2 * math.pi
5 print(f"Area of a circle with radius 3 = {area}")
6
7 # import functions of the package separately
8 from math import sin, cos, pi, sqrt
9
10 angle = 23/180*pi
11 formula = sqrt(2/3) * sin(angle) + cos(angle)
12
13 # import packages or functions and rename them
14 from math import sqrt as sr
15
16 print(f"The square root of 2 = {sr(2)}")

```

2 Tidy and Documented code

2.1 Tidy Writing style

- Writing **readable** code
- Clear and not too long variable names
- Use spaces in a consistent manner
- Use a Python code formatter such as **Black**:
<https://github.com/psf/black>
- Document code both with **comments** and **docstrings**

2.2 Comment lines

- Text of a line after a hash symbol is ignored
- The hash symbol can be in the beginning of the line:

```
# A comment on a single line
```

Comments can explain the next line of code

```
# Convert Matlab-indices to zero-based  
ind = m_index - 1
```

Or the comment can start after a statement

```
y = 12 + x # An inline comment: a hash symbol after 2 spaces
```

2.3 Multi-line comments]

Multi-line strings can be used as comments, but

- they are not meant as comments (not ignored)
- they can become docstrings (documentation-strings)

```
'''  
This multi-line string is not assigned to a  
variable, therefore doesn't influence your code  
directly  
'''
```

2.4 Docstrings

- Docstrings are multi-line strings providing documentation
- They populate the `__doc__` variable
- The `help()` function uses the `__doc__` variable

```
import math
print(math.__doc__)
```

This module provides access to the mathematical functions defined by the C standard.

```
import math
help(math.sin)
```

Help on built-in function sin in module math:

```
sin(x, /)
    Return the sine of x (measured in radians).
```

- We will revisit Docstrings for functions and classes later.

3 Program Errors

3.1 Different error-types

Different types of errors can be encountered

- **Syntax errors:** code inconsistent with the Python language, the code will not run (i.e. not start).
- **Runtime errors:** exceptions occurring while the program runs, the code will crash.
- **Semantic errors:** the code does not what was intended

3.1.1 Syntax error examples

- Usage of unknown identifiers (variable, function, class)

```
positive_number = uint(45)
```

```
NameError: name 'uint' is not defined
```

```
-----  
NameError                                Traceback (most recent call last)  
Cell In[36], line 1  
----> 1 positive_number = uint(45)  
NameError: name 'uint' is not defined
```

- Misaligned indentation

```
a = 34  
b = 5  
print(f"The sum is {a+b}")
```

```
IndentationError: unexpected indent (3752930843.py, line 2)
```

```
Cell In[37], line 2  
    b = 5  
    ^  
IndentationError: unexpected indent
```

- Mistaken symbol ":" instead of ","

```
a = 3: b = 5
```

```
SyntaxError: invalid syntax (173182802.py, line 1)
```

```
Cell In[38], line 1  
    a = 3: b = 5  
    ^  
SyntaxError: invalid syntax
```

3.1.2 Runtime error examples

- Division by zero

```
fraction = 1 / 0
```

```
ZeroDivisionError: division by zero
```

```
-----  
ZeroDivisionError                                Traceback (most recent call last)  
Cell In[39], line 1  
----> 1 fraction = 1 / 0  
ZeroDivisionError: division by zero
```

- Type mismatch

```
netto_price = 230.50  
kdv_ratio = 0.21  
print("Price: " + netto_price + " + " + kdv_ratio + " kdv")
```

```
TypeError: can only concatenate str (not "float") to str
```

```
-----  
TypeError                                Traceback (most recent call last)  
Cell In[40], line 3  
      1 netto_price = 230.50  
      2 kdv_ratio = 0.21  
----> 3 print("Price: " + netto_price + " + " + kdv_ratio + " kdv")  
TypeError: can only concatenate str (not "float") to str
```

Knuth, Donald E. 1984. "Literate Programming." *Comput. J.* 27 (2): 97–111. <https://doi.org/10.1093/comjnl/27.2.97>.